Universidad Mariano Gálvez De Guatemala

San Pedro Sacatepéquez, San Marcos



Documentación Técnica

Danna Monserrath Yorlen Díaz Hernández	0903-24-6624
Fátima Yamileth Pérez Fuentes	0903-24-12439
Jackeline Nayeli Chay Flores	0903-24-7500
Edison Roberto Chilel Pérez	0903-23-17834
Carlos Emilio Jerónimo Salvador	0903-24-24272

Estudiantes

Sexto Semestre de Ingeniería en Sistemas

Sección: “A”

Ingeniero Emmanuel Mazariegos

Docente

San Pedro Sacatepéquez, San Marcos, 31 de octubre de 2026

Índice

1. Introducción 4

2. Objetivos 5

2.1. Objetivo general 5

2.2. Objetivos específicos 5

3. Arquitectura del sistema 6

3.1. Componentes principales 6

3.2. Organización lógica 6

3.3. Archivos y módulos principales 7

4. Algoritmos utilizados 8

4.1. Simulación de autómatas finitos 8

4.2. Conversión de AFN a AFD 8

4.3. Minimización de autómatas 9

4.4. Validación de expresiones regulares 10

4.5. Búsqueda de coincidencias 11

4.6. Generación de autómatas desde expresiones regulares 11

4.7. Registro de eventos 12

5.1. Python 13

5.2. Tkinter 13

5.3. Expresiones regulares de Python 13

5.4. SQLite 13

5.5. Git y GitHub 14

6. Estructura y comunicación entre módulos 14

6.1. Interfaz gráfica 14

6.2. Procesamiento de autómatas 15

6.3. Procesamiento de expresiones regulares 15
6.4. Gestión de registros 15
6.5. Base de datos local 15
7. Requisitos técnicos y dependencias 16
7.1. Requisitos de software 16
8. Instalación y puesta en funcionamiento 16
8.1. Obtener el código fuente 16
9. Manejo de errores y excepciones 17
9.1. Errores de expresiones regulares 17
9.2. Errores de entrada 17
10. Seguridad e integridad de la información 17
12. Conclusiones 19

1. Introducción

El proyecto consiste en el desarrollo de una aplicación de escritorio que permite trabajar con autómatas finitos y expresiones regulares mediante una interfaz gráfica. Su finalidad es facilitar la comprensión y aplicación de conceptos relacionados con la teoría de autómatas, el reconocimiento de cadenas y el procesamiento de patrones.

La aplicación integra las funcionalidades del Sistema de Autómatas Finitos con el módulo Regex Lab, procurando mantener la estructura del sistema existente e incorporar nuevas herramientas para validar cadenas, generar autómatas a partir de expresiones regulares y registrar las actividades realizadas.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación que integre herramientas para la creación, simulación, conversión y minimización de autómatas finitos, así como el análisis y la validación de expresiones regulares mediante una interfaz gráfica.

2.2. Objetivos específicos

- Permitir la creación y configuración de autómatas finitos.
- Simular el procesamiento de cadenas mediante AFN y AFD.
- Implementar la conversión de un AFN a un AFD y la minimización de autómatas.
- Validar cadenas completas y buscar coincidencias mediante expresiones regulares.
- Incorporar un generador de autómatas a partir de expresiones regulares.
- Registrar los eventos relevantes del sistema para facilitar su seguimiento y revisión.

3. Arquitectura del sistema

La aplicación se organiza en módulos que separan la interfaz gráfica, la lógica de los autómatas, el procesamiento de expresiones regulares y el almacenamiento de registros.

3.1. Componentes principales

Interfaz gráfica: permite al usuario interactuar con las funciones del sistema mediante ventanas, pestañas, campos de texto y botones.

Motor de autómatas: contiene las estructuras y operaciones necesarias para representar estados, alfabetos, transiciones, estados iniciales y estados finales.

Motor de expresiones regulares: procesa los patrones ingresados y permite validar cadenas o buscar coincidencias en un texto.

Módulo de conversión y minimización: ejecuta los procedimientos necesarios para transformar autómatas y reducir sus estados cuando corresponde.

Módulo de registros: administra los eventos producidos durante el uso de Regex Lab, incluidos los errores y las advertencias que se hayan instrumentado.

Base de datos: utiliza SQLite para almacenar los registros del sistema y facilitar su consulta posterior.

3.2. Organización lógica

El funcionamiento general se puede representar de la siguiente manera:

Usuario → Interfaz gráfica → Módulo seleccionado → Procesamiento → Resultado mostrado al usuario.

En las operaciones de Regex Lab que generan registros, el procesamiento también comunica los eventos al módulo de registros, que los almacena en la base de datos.

3.3. Archivos y módulos principales

Archivo o modulo	Función
main.py	Punto de entrada de la aplicación.
automatas_finitos.py	Interfaz del sistema de autómatas finitos.
interfaz.py	Interfaz gráfica de Regex Lab.
automatas.py	Estructuras y operaciones básicas de los autómatas, según la organización del proyecto.
conversion.py	Operaciones de conversión de AFN a AFD.
minimizacion.py	Procedimientos de minimización de autómatas.
simulador.py y simulador_afd.py	Simulación de autómatas, según el tipo implementado.
regex_engine.py	Validación y búsqueda de patrones mediante expresiones regulares.

database.py	Inicialización y consultas de la base de datos de registros.
log_manager.py	Administración y exportación de registros.
requirements.txt	Dependencias necesarias para ejecutar el proyecto.

---

4. Algoritmos utilizados

4.1. Simulación de autómatas finitos

El simulador recibe una cadena y procesa sus símbolos de acuerdo con las transiciones definidas en el autómata.

Procedimiento general:

1. Identificar el estado inicial.
2. Leer la cadena de entrada.
3. Procesar cada símbolo según las transiciones disponibles.
4. Determinar el estado o conjunto de estados alcanzables.
5. Comprobar si el resultado cumple las condiciones de aceptación.
6. Mostrar si la cadena es aceptada o rechazada.

En un AFD, cada estado tiene como máximo una transición definida para cada símbolo del alfabeto. En un AFN, un símbolo puede conducir a varios estados posibles.

4.2. Conversión de AFN a AFD

La conversión permite construir un autómata determinista equivalente a uno no determinista.

El procedimiento habitual, conocido como construcción por subconjuntos, consiste en:

1. Tomar el estado inicial del AFN, considerando los cierres-ε cuando existan.
2. Representar cada conjunto de estados del AFN como un estado del AFD.
3. Calcular los estados alcanzables para cada símbolo del alfabeto.
4. Incorporar los nuevos conjuntos de estados encontrados.
5. Repetir el proceso hasta que no existan nuevos conjuntos por procesar.
6. Identificar los estados finales del AFD a partir de los conjuntos que contengan estados finales del AFN.

El resultado es un autómata determinista que reconoce el mismo lenguaje que el AFN original.

4.3. Minimización de autómatas

La minimización busca obtener un AFD equivalente con el menor número posible de estados, manteniendo el lenguaje reconocido.

El procedimiento general incluye:

1. Identificar los estados alcanzables desde el estado inicial.
2. Separar inicialmente los estados finales de los no finales.

3. Analizar si los estados de cada grupo se comportan de manera equivalente ante los símbolos del alfabeto.
4. Dividir los grupos cuando se detecten diferencias.
5. Repetir el análisis hasta que los grupos ya no necesiten subdividirse.
6. Construir el autómata reducido.

La técnica concreta utilizada debe corresponder a la implementación del módulo de minimización del proyecto.

4.4. Validación de expresiones regulares

Regex Lab permite ingresar una expresión regular y una cadena para comprobar si esta cumple el patrón establecido.

El procedimiento consiste en:

1. Leer la expresión regular ingresada.
2. Comprobar que la sintaxis sea válida.
3. Recibir la cadena que se desea evaluar.
4. Aplicar el patrón mediante las funciones de expresiones regulares de Python.
5. Mostrar el resultado de aceptación o rechazo.
6. Registrar el evento correspondiente cuando la operación esté configurada para generar logs.

Para la validación completa se debe utilizar una operación que compruebe que toda la cadena coincide con el patrón, no únicamente una parte de ella.

4.5. Búsqueda de coincidencias

Esta función identifica las partes de un texto que cumplen con una expresión regular.

Procedimiento:

1. Recibir el patrón y el texto.
2. Verificar que el patrón sea válido.
3. Buscar las coincidencias en el contenido.
4. Recuperar los fragmentos encontrados.
5. Mostrar los resultados al usuario.

La búsqueda de coincidencias es diferente de la validación completa: una cadena puede contener un fragmento válido aunque el texto entero no cumpla el patrón.

4.6. Generación de autómatas desde expresiones regulares

El módulo generador permite trabajar con la transformación de una expresión regular en una representación de autómata.

Una estrategia habitual es utilizar la construcción de Thompson para obtener un AFN a partir de una expresión regular. Posteriormente, el AFN puede convertirse en AFD y minimizarse mediante los módulos correspondientes.

Importante: esta sección debe describir los pasos que realmente ejecuta el generador del proyecto. Si alguna transformación todavía no está implementada o disponible desde la interfaz, debe indicarse como trabajo pendiente.

4.7. Registro de eventos

El sistema de registros permite conservar información relevante sobre las operaciones realizadas.

Los eventos pueden incluir:

- Validaciones de cadenas.
- Búsquedas de coincidencias.
- Generación o transformación de autómatas.
- Advertencias.
- Errores de procesamiento.
- Consultas y exportaciones de registros.

La consulta de registros facilita revisar las actividades realizadas y detectar posibles problemas.

5. Tecnologías y herramientas de desarrollo

Para el desarrollo del Sistema de Autómatas Finitos y Regex Lab se utilizaron herramientas de programación, componentes gráficos y mecanismos de almacenamiento que permiten integrar las funcionalidades del proyecto.

5.1. Python

Python es el lenguaje de programación utilizado para desarrollar la lógica de la aplicación. Permite implementar estructuras de datos, procesamiento de cadenas, operaciones con autómatas y manejo de excepciones.

Su utilización facilita la organización del sistema en módulos independientes, favoreciendo la reutilización del código y el mantenimiento de las funcionalidades existentes.

5.2. Tkinter

Tkinter se utiliza para construir la interfaz gráfica de escritorio. Mediante sus componentes es posible presentar ventanas, pestañas, botones, campos de entrada, tablas y áreas de resultados.

La interfaz funciona como medio de comunicación entre el usuario y los módulos encargados del procesamiento de autómatas y expresiones regulares.

5.3. Expresiones regulares de Python

El procesamiento de expresiones regulares se realiza mediante las herramientas disponibles en Python para trabajar con patrones de texto. Estas permiten validar cadenas, identificar coincidencias y detectar patrones que no cumplen con la sintaxis requerida.

El uso de estas herramientas evita tener que desarrollar desde cero un motor completo para interpretar expresiones regulares.

5.4. SQLite

SQLite se utiliza como sistema de almacenamiento para los registros de eventos de la aplicación. Permite conservar información en una base de datos local sin requerir la instalación de un servidor de base de datos independiente.

El módulo encargado de la base de datos administra su inicialización y las consultas necesarias para recuperar información almacenada.

5.5. Git y GitHub

Git se utilizó para el control de versiones del código fuente, mientras que GitHub permitió centralizar el repositorio y facilitar el trabajo colaborativo.

La persona responsable de crear el repositorio agregó a los demás integrantes como colaboradores. Cada integrante descargó la carpeta del proyecto, realizó modificaciones en su equipo y compartió sus avances mediante git push.

Este mecanismo permitió mantener un historial de cambios y reunir las contribuciones realizadas durante el desarrollo.

6. Estructura y comunicación entre módulos

La organización modular permite separar las responsabilidades de la aplicación y facilita la localización de errores durante el mantenimiento.

6.1. Interfaz gráfica

La interfaz recibe los datos ingresados por el usuario y solicita la ejecución de las operaciones correspondientes. Posteriormente, presenta los resultados obtenidos por los módulos de procesamiento.

Las validaciones de entrada y el manejo de errores deben evitar que datos incorrectos provoquen fallos innecesarios en la aplicación.

6.2. Procesamiento de autómatas

El módulo de autómatas administra los elementos que conforman cada estructura: estados, alfabeto, estado inicial, estados finales y transiciones.

A partir de estos elementos se realizan las operaciones de simulación y, cuando están disponibles, los procesos de conversión y minimización.

6.3. Procesamiento de expresiones regulares

Este componente recibe los patrones y las cadenas que se desean evaluar. Comprueba la validez del patrón, ejecuta la operación solicitada y devuelve el resultado para su presentación en la interfaz.

Es importante diferenciar entre un patrón sintácticamente incorrecto y una cadena que simplemente no coincide con un patrón válido.

6.4. Gestión de registros

El módulo de registros comunica los eventos relevantes al componente de almacenamiento. Según la instrumentación implementada, puede registrar operaciones realizadas, advertencias y errores.

La separación entre la lógica de procesamiento y el registro de eventos permite modificar la forma de almacenar o consultar los registros sin alterar necesariamente los algoritmos principales.

6.5. Base de datos local

La base de datos SQLite conserva los eventos que la aplicación tiene configurados para almacenar. El módulo correspondiente debe encargarse de inicializar la estructura de almacenamiento y ejecutar consultas controladas.

7. Requisitos técnicos y dependencias

Para ejecutar la aplicación se necesita un entorno compatible con la versión de Python utilizada durante el desarrollo.

7.1. Requisitos de software

- Sistema operativo compatible con la versión de Python instalada.
- Python con soporte para los módulos utilizados por el proyecto.
- Tkinter disponible en la instalación de Python.
- Permisos de lectura y escritura en la ubicación donde se almacenen la base de datos y los archivos de registros.

8. Instalación y puesta en funcionamiento

Este apartado describe el procedimiento técnico para preparar el entorno y ejecutar la aplicación desde el código fuente.

8.1. Obtener el código fuente

La aplicación puede obtenerse descargando la aplicación ejecutable en el siguiente repositorio; https://github.com/Dannadx/Sistema_Automatas.git

La aplicación aparece en reléase v2.1.0 con el nombre de Sistema de Autómatas Finitos y Regex Lab, por lo que se debe buscar el archivo SistemaAutomatas.exe, descargar y ejecutar.

Listo, la aplicación ya estará funcionando.

9. Manejo de errores y excepciones

El manejo de errores es importante para evitar interrupciones inesperadas y facilitar la identificación de problemas durante la ejecución.

9.1. Errores de expresiones regulares

Cuando el usuario introduce un patrón con sintaxis incorrecta, el sistema el error y muestra un mensaje.

9.2. Errores de entrada

Los datos incompletos, los símbolos no permitidos o las cadenas que no cumplen las condiciones establecidas provocan un error.

Una cadena rechazada por un autómata o que no coincide con una expresión regular no constituye, por sí misma, un error de ejecución.

10. Seguridad e integridad de la información

Las principales medidas recomendadas son:

- Validar las entradas antes de procesarlas.
- Controlar las excepciones producidas por operaciones de archivos y bases de datos.

- Utilizar consultas parametrizadas cuando se interactúe con SQLite.
- Evitar modificar manualmente los archivos internos de la base de datos mientras la aplicación se encuentra en funcionamiento.

Estas medidas deben aplicarse de acuerdo con las funcionalidades y los mecanismos de seguridad que realmente tenga implementados el proyecto.

12. Conclusiones

El desarrollo del Sistema de Autómatas Finitos y Regex Lab permite integrar conceptos de teoría de autómatas y expresiones regulares en una aplicación de escritorio. La separación de responsabilidades entre la interfaz, los módulos de procesamiento y el almacenamiento contribuye a organizar el código y facilita su mantenimiento.

La implementación de las operaciones con autómatas permite aplicar conceptos relacionados con estados, transiciones, reconocimiento de cadenas, conversión y minimización. Por su parte, el procesamiento de expresiones regulares permite trabajar con patrones de texto y diferenciar entre validación completa y búsqueda de coincidencias.

La incorporación de una base de datos local para los registros proporciona un mecanismo para consultar los eventos que la aplicación tenga configurados. Esto contribuye al seguimiento de las operaciones y facilita la identificación de posibles problemas.

Finalmente, el uso de Git y GitHub favorece el control de versiones y la colaboración entre los integrantes del equipo. La documentación técnica constituye una referencia para comprender la organización del sistema, preparar su entorno de ejecución y realizar futuras modificaciones.